

Ejercicio 2: Oferta de Compra Eficiente (Algoritmo Voraz)

Enunciado

En una tienda hay una oferta. Si compras un producto, puedes llevarte gratis otro cualquiera de precio inferior o igual. Necesitas esta lista de productos:

$a_1, a_2, \dots, a_n$, y quieres gastar lo mínimo posible.

Diseña un algoritmo voraz que escriba en pantalla qué productos compras y cuáles te llevas gratis, demuestra su corrección y justifica su coste en tiempo.

1. Estrategia Voraz (Greedy)

La oferta nos permite emparejar productos de dos en dos (uno comprado y uno gratis), siempre que el producto gratis sea de precio menor o igual al comprado.

Para minimizar el gasto total, queremos **maximizar el valor de los productos que nos llevamos gratis**. La idea intuitiva (y óptima) es la siguiente:

1. **Ordenar** la lista de precios de los productos de mayor a menor.
2. Recorrer la lista ordenada y aplicar una estrategia de **emparejamiento descendente**:
 - El producto más caro de todos los disponibles lo tenemos que comprar obligatoriamente (no hay ningún producto más caro que nos lo pueda dar gratis).
 - Para aprovechar esta compra al máximo, nos llevamos gratis el **siguiente producto más caro disponible** (que por definición tiene un precio menor o igual).
 - Repetimos este proceso con los productos restantes.

Ejemplo Visual:

Si queremos los productos con precios: $\{10, 20, 5, 12, 8, 3\}$

1. Ordenamos de mayor a menor: 20, 12, 10, 8, 5, 3
 2. **Primer par**: Compras 20, te llevas gratis 12.
 3. **Segundo par**: Compras 10, te llevas gratis 8.
 4. **Tercer par**: Compras 5, te llevas gratis 3.
- **Total pagado**: $20 + 10 + 5 = 35$ (ahorro de $12 + 8 + 3 = 23$).

2. Algoritmo en Pseudocódigo

```
Algoritmo OfertaTienda(Precios, n)
    // Entrada: Un array Precios[1..n] con los precios de los productos
    // Salida: Listado de productos comprados y productos gratis

    // 1. Ordenar de mayor a menor
    Ordenar_Descendente(Precios)
```

```

Escribir "--- PLAN DE COMPRA ÓPTIMO ---"

i ← 1
Mientras (i ≤ n) hacer
    // El elemento actual i es el más caro restante, se compra obligatoriamente
    Escribir "COMPRAR: Producto de precio ", Precios[i]

    // Si hay un elemento posterior, nos lo llevamos gratis
    Si (i + 1 ≤ n) entonces
        Escribir "GRATIS: Producto de precio ", Precios[i + 1]
    FinSi

    // Avanzamos de 2 en 2
    i ← i + 2
FinMientras
FinAlgoritmo

```

3. Demostración de Corrección (Argumento de Intercambio)

Supongamos que existe una solución óptima O que es distinta de nuestra solución voraz V . Queremos demostrar que podemos transformar O en V sin aumentar el coste total.

Sea $P = (p_1, p_2, \dots, p_n)$ la lista de precios ordenada de mayor a menor ($p_1 \geq p_2 \geq \dots \geq p_n$).

- La solución voraz V empareja obligatoriamente a los elementos adyacentes: (p_1, p_2) , (p_3, p_4) , $\dots$. En cada pareja, el primero se paga y el segundo es gratis. El ahorro es $\sum p_{\text{pares}}$.

Si O no coincide con V , significa que existe algún elemento p_i (comprado) que se empareja con un elemento p_j (gratis) tal que no son adyacentes en la lista ordenada. Analicemos el primer desfase:

Sean dos parejas en O :

- (p_a, p_c) donde pagamos p_a y nos llevamos gratis p_c .
- (p_b, p_d) donde pagamos p_b y nos llevamos gratis p_d .

Supongamos que el orden de precios es $p_a \geq p_b \geq p_c \geq p_d$ (es decir, están "cruzados" en lugar de estar emparejados adyacentemente como (p_a, p_b) y (p_c, p_d)).

- El coste de estas dos parejas en O es: $p_a + p_b$.
- El ahorro obtenido en O por estas dos parejas es: $p_c + p_d$.

Si intercambiamos los emparejamientos para hacerlos de forma voraz (unir los más caros entre sí y los más baratos entre sí):

- Nueva Pareja 1: (p_a, p_b) - Pagamos p_a , gratis p_b (es válido porque $p_a \geq p_b$).
- Nueva Pareja 2: (p_c, p_d) - Pagamos p_c , gratis p_d (es válido porque $p_c \geq p_d$).
- Nuevo coste:** $p_a + p_c$.

- **Nuevo ahorro:** $p_b + p_d$.

Comparación de costes:

Como $p_b \geq p_c$ (por estar ordenados), se cumple que:

$$\text{Coste Voraz} \leq \text{Coste de } O \implies (p_a + p_c) \leq (p_a + p_b)$$

Al deshacer el cruce y emparejar de forma adyacente descendente, el coste de la compra disminuye (o se mantiene igual). Repitiendo este proceso de intercambio para todos los elementos no adyacentes, transformamos la solución óptima O en la solución voraz V sin aumentar el coste en ningún paso. Esto demuestra que **la solución voraz es óptima**.

4. Análisis de Coste en Tiempo

- **Ordenación:** El paso dominante del algoritmo es ordenar los n productos. Utilizando un algoritmo de ordenación eficiente como *MergeSort* o *QuickSort*, el coste es de $\mathcal{O}(n \log n)$.
- **Bucle de emparejamiento:** Recorrer la lista ya ordenada y agrupar los elementos de dos en dos se realiza en un único bucle lineal con un coste de $\mathcal{O}(n)$.
- **Coste Total:** $\mathcal{O}(n \log n)$ en tiempo.
- **Espacial:** $\mathcal{O}(1)$ si la ordenación se realiza *in-place* sobre el vector original, o $\mathcal{O}(n)$ si se requiere almacenar la salida o copias auxiliares.